

Course Q&A Chatbot – Architecture (RAG + Citations)

Overview

The Course Q&A Chatbot enables students to query course material and receive context-aware, citation-based answers. It leverages Retrieval-Augmented Generation (RAG) with embeddings and a vector database to improve accuracy and reduce hallucination.

System Architecture

1. **React Frontend**
 - Provides the user interface.
 - Handles input, displays responses with citations.
 - Manages state and communicates with backend APIs.
2. **Express Backend (Node.js/TypeScript)**
 - Exposes REST API endpoints for the frontend.
 - Handles user queries, embeddings, and retrieval logic.
 - Connects to pgvector for similarity search and Ollama LLM for embeddings and generation.
 - Organized into routes, controllers, and services for modularity.
3. **Postgres + pgvector**
 - Stores course content as chunks with embeddings.
 - Provides efficient similarity search using cosine or L2 distance.
 - Supports metadata storage and citation references.
4. **Ollama LLM (Local Embedding + Generation)**
 - Generates embeddings for documents and queries.
 - Synthesizes final answers from retrieved chunks.
 - Ensures generated responses include citations from the original content.

Data Flow

1. **Ingestion:** Course documents are parsed, chunked, embedded, and stored in Postgres with pgvector.
2. **Query:** A student asks a question through the React frontend.
3. **Retrieval:** The backend embeds the query, performs similarity search in pgvector, and retrieves top-k relevant chunks.
4. **Answer Generation:** The retrieved chunks are passed to Ollama LLM, which generates a concise, citation-backed answer.
5. **Response:** The answer and citations are returned to the frontend for display.

Technology Stack

- **Frontend:** React.js
- **Backend:** Node.js with Express
- **Database:** Postgres with pgvector extension
- **LLM:** Ollama (local embeddings and text generation)
- **Evaluation:** Python scripts for recall@5, faithfulness, and latency measurement

Key Features

- Semantic search using embeddings for meaning-based retrieval.
- RAG pipeline with citations for accuracy and reliability.
- Local LLM for embedding and generation (no external API dependency).

- Evaluation metrics for quality and performance.
- Extendable to hybrid retrieval and re-ranking approaches.

Reference

The detailed architecture diagram is included in the repository under docs/architecture.png.